

Darra AI — GMSC Modeling Report

From Preprocessing Pipeline to SHAP Explainability

Dataset: Give Me Some Credit (150,000 borrowers) · **Target:** `SeriousDlqin2yrs` (6.68% positive, ≈14 : 1)

Primary metric: AUC-PR (average precision) · **Random state:** 42

1. Summary

Item	Result
Final model	XGBoost pipeline (median imputation → XGBoost) + isotonic calibration
Baseline	Logistic Regression (class-weighted)
CV AUC-PR (5-fold OOF)	LR 0.3884 → XGBoost 0.3977
CV ROC-AUC (5-fold OOF)	LR 0.8561 → XGBoost 0.8619
Calibration (Brier, train OOF)	0.1438 → 0.0492
Threshold (target recall 80%)	0.0593
Final test	AUC-PR 0.3904 · ROC-AUC 0.8647 · Recall 0.8185 · Precision 0.1917 · Brier 0.0489
Top risk drivers (SHAP)	Late payments, credit utilization, age

Test performance matches cross-validation performance, so there is no sign of overfitting.

2. Data split

Set	Rows	Default rate
Train	120,000	6.68%
Test	30,000	6.68%

- Stratified split on the target (80/20, `random_state=42`).
- The test set is used **once**, in the final evaluation (Section 8).
- There is no separate validation set: calibration and the threshold are fit on **out-of-fold (OOF) predictions of the training set** (see Section 7 and the deviations in Section 11).

3. Feature engineering (`build_gmsc_features`)

The function reads the raw file and applies the cleaning rules from the EDA, then builds the model features. All steps are row-wise (no statistics are learned from the data), except where noted in Section 4.

Rule	What it does
Placeholder codes	96/98 in the three past-due columns → NaN + <code>derived_placeholder_code_flag</code> (269 rows)
Age	<code>age = 0</code> → NaN
Income	<code>income_missing_flag</code> ; <code>derived_log_income = log1p(income)</code>
Debt ratio	Cleaned <code>DebtRatio</code> is NaN when income is missing or 0; <code>derived_debt_ratio_outlier = DebtRatio > 5</code> ; <code>derived_overspend_flag = DebtRatio > 1</code>
Obligations	<code>derived_obligations = DebtRatio × income</code> (only when income > 0 and DebtRatio ≤ 5)
Surplus	<code>derived_monthly_surplus = income - obligations</code>
Utilization	<code>derived_utilization_capped = min(utilization, 1)</code> ; <code>derived_utilization_outlier_flag = utilization > 1</code>

Rule	What it does
Late payments	<code>derived_late_count</code> = sum of the three past-due counters (after fixing 96/98); <code>derived_late_severity</code> = $1 \times (30-59) + 2 \times (60-89) + 3 \times (90+)$
Dependents	<code>dependents_missing_flag</code> ; missing filled with 0
Duplicates	<code>duplicate_flag</code> (609 duplicate rows flagged, not used as a feature and not removed)

Model features (15):

Group	Features
Continuous (6)	<code>age</code> , <code>derived_log_income</code> , <code>derived_obligations</code> , <code>derived_monthly_surplus</code> , <code>derived_expense_ratio</code> , <code>derived_utilization_capped</code>
Counts (3)	<code>dependents</code> , <code>derived_late_count</code> , <code>derived_late_severity</code>
Flags (6)	<code>income_missing_flag</code> , <code>dependents_missing_flag</code> , <code>derived_placeholder_code_flag</code> , <code>derived_debt_ratio_outlier</code> , <code>derived_overspend_flag</code> , <code>derived_utilization_outlier_flag</code>

Several features are derived from the same raw inputs (income and debt ratio feed 4-5 features). This matters for scenarios (Section 10) and for reading SHAP (Section 9).

4. Preprocessing pipeline

Missing values in the training features: `derived_log_income` 23,675 · `derived_obligations` 25,697 · `derived_monthly_surplus` 25,697 · `derived_expense_ratio` 24,997 · `age` 1.

Steps:

1. **Clipping at the 1st / 99th percentile** of the numeric features. Limits are computed on the train set and applied unchanged to the test set. Upper limits: age 87, log-income 10.13, obligations 9,096.5, surplus 20,480.1, expense ratio 3.00, utilization 1.0, dependents 4, late count 5, late severity 10. Lower limit of surplus: -1,682.7 (others 0, age 24).
2. **Median imputation** for the numeric features (fit inside every CV fold).
3. **Standard scaling** (Logistic Regression only; XGBoost does not need it).
4. **Flags** pass through unchanged.

```
ColumnTransformer
├─ numeric (9) → SimpleImputer(median) [→ StandardScaler for LR]
├─ flags (6) → passthrough
└─
  ↓
Model (LogisticRegression | XGBClassifier)
```

5. Baseline — Logistic Regression

`LogisticRegression(class_weight="balanced")`, 5-fold StratifiedKFold.

Metric	CV mean
AUC-PR	0.3888
ROC-AUC	0.8561

Coefficients (standardized features):

Feature	Coef.	Feature	Coef.
<code>placeholder_code_flag</code>	+2.573	<code>debt_ratio (expense)</code>	-0.066
<code>utilization_capped</code>	+0.640	<code>dependents_missing_flag</code>	-0.099
<code>late_severity</code>	+0.448	<code>monthly_surplus</code>	-0.218
<code>overspend_flag</code>	+0.376	<code>age</code>	-0.249
<code>utilization_outlier_flag</code>	+0.359	<code>income_missing_flag</code>	-0.293
<code>late_count</code>	+0.340		

Feature	Coef.	Feature	Coef.
obligations	+0.187		
debt_ratio_outlier	+0.159		
log_income	+0.066		
dependents	+0.021		

The direction of every coefficient is sensible (more lateness and utilization → more risk; older age and higher surplus → less risk). The placeholder (96/98) flag has by far the largest coefficient, consistent with the EDA (55% default rate in that group).

6. Main model — XGBoost + Optuna

- **Class imbalance:** `scale_pos_weight = 13.96` (negatives / positives in train).
- **Search:** Optuna (TPE sampler, seed 42), **40 trials**, objective = mean AUC-PR over 5-fold `StratifiedKfold`.
- **Inside every fold:** the imputer is fit on the train fold only; early stopping (50 rounds, `aucpr`) uses the fold's validation part; the mean best iteration becomes the final `n_estimators`.
- `StratifiedGroupKfold` is not needed: there is no customer ID (the row index is dropped).

Best parameters:

Parameter	Value
max_depth	4
learning_rate	0.0149
subsample	0.838
colsample_bytree	0.991
min_child_weight	9
reg_lambda	0.765
n_estimators (from early stopping)	482

Best CV AUC-PR: 0.3991.

Baseline vs tuned XGBoost (same 5 folds, OOF predictions)

Model	AUC-PR	ROC-AUC
Logistic Regression	0.3884	0.8561
Tuned XGBoost	0.3977	0.8619
Random classifier (base rate)	0.0668	0.5

Justification: XGBoost is better on both metrics, but the gain is small (+0.009 AUC-PR). Both models are $\approx 6\times$ better than random on AUC-PR. XGBoost is kept as the main model because it handles the skewed features and interactions without manual transformation and works directly with SHAP; Logistic Regression remains a transparent fallback.

7. Calibration and threshold

`scale_pos_weight` inflates the raw scores, so they are not real probabilities.

Method: the OOF XGBoost scores from Section 6 are used as the "validation" scores. An **isotonic regression** is fit on them. To measure its effect honestly, the calibrated probabilities used for the metrics, the curve and the threshold are **cross-fitted** (each part calibrated by an isotonic fit on the other parts).

Metric (train OOF)	Raw XGBoost	Isotonic
Brier score	0.1438	0.0492

Metric (train OOF)	Raw XGBoost	Isotonic
AUC-PR	0.3977	0.3925

The Brier score improves by 66%. AUC-PR drops slightly (-0.005) because isotonic regression produces a step function with ties; ranking quality is essentially preserved.

Threshold: the highest threshold that still reaches **80% recall** (business choice) on the OOF calibrated scores:

	Value
Threshold	0.0593
OOF Recall	0.803
OOF Precision	0.195

8. Final test evaluation (run once)

Final model: XGBoost fit on the **full training set** with the tuned parameters; isotonic calibrator fit on all training OOF scores; the test set is clipped with the **train** limits.

Metric	Value
AUC-PR	0.3904 (base rate 0.0668)
ROC-AUC	0.8647
Brier (calibrated)	0.0489 (raw: 0.1443)
Threshold	0.0593
Recall	0.8185
Precision	0.1917
F1	0.3107

Confusion matrix

	Predicted 0	Predicted 1
Actual 0	21,077	6,918
Actual 1	364	1,641

Reading it:

- The model catches **1,641 of 2,005 defaulters (81.9%)** and misses 364.
- It flags **8,559 customers (28.5% of all customers)**. Only 19.2% of flagged customers are real defaulters, i.e. **2.9x the base rate** (6.7%).
- 6,918 good customers are flagged (false-positive rate 24.7%).
- **Trade-off:** raising recall to 82% costs precision of 19%. This is acceptable for an early-warning product (a flag triggers advice, not a rejection), but must be stated clearly. A higher threshold gives better precision and lower recall.

Test results are consistent with CV (AUC-PR $0.3977 \rightarrow 0.3904$, ROC-AUC $0.8619 \rightarrow 0.8647$), so the model generalizes.

9. Explainability — SHAP

`shap.TreeExplainer` on the final XGBoost model. SHAP values are in **log-odds of the raw model score** (before calibration): they show which features push risk up or down and by how much. The risk % shown to a user is the calibrated probability. A 5,000-row random sample of the test set was used for the global plots.

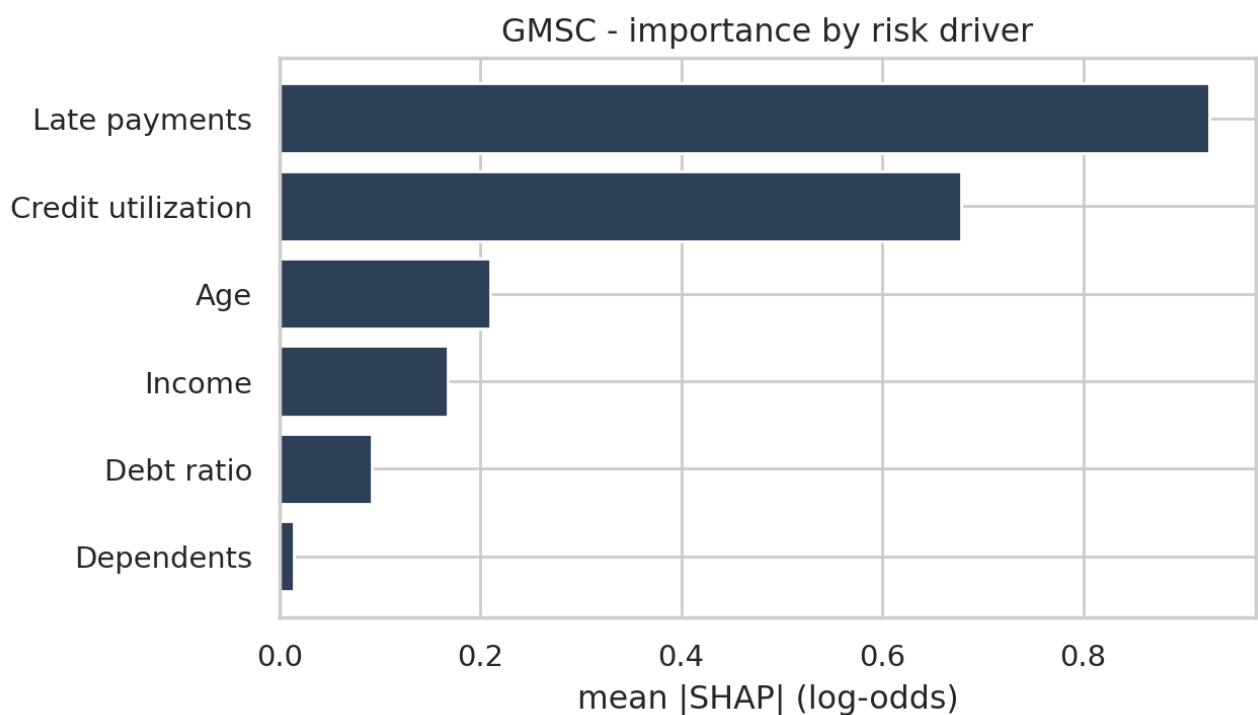
9.1 Global importance

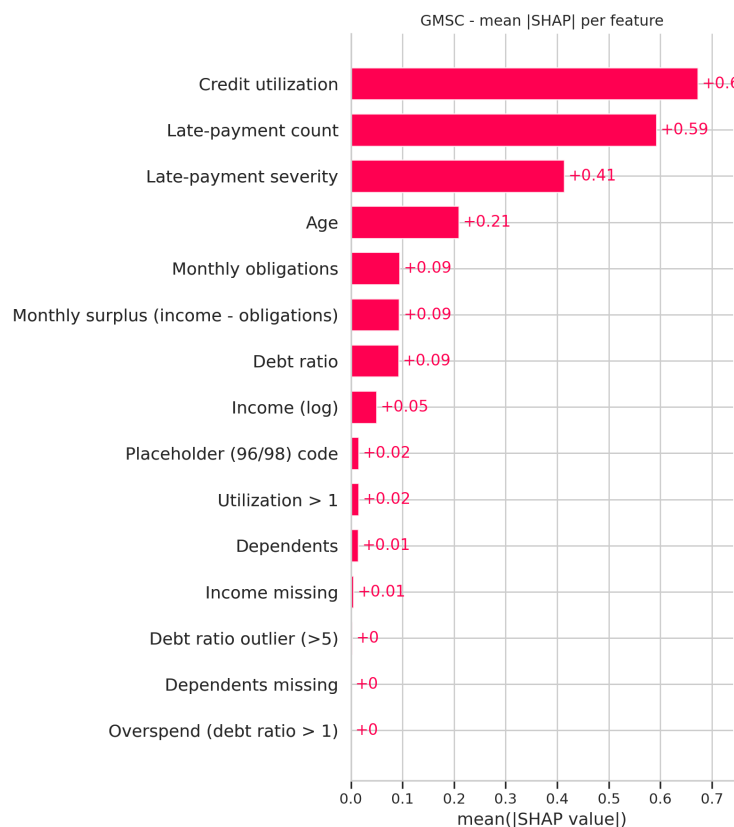
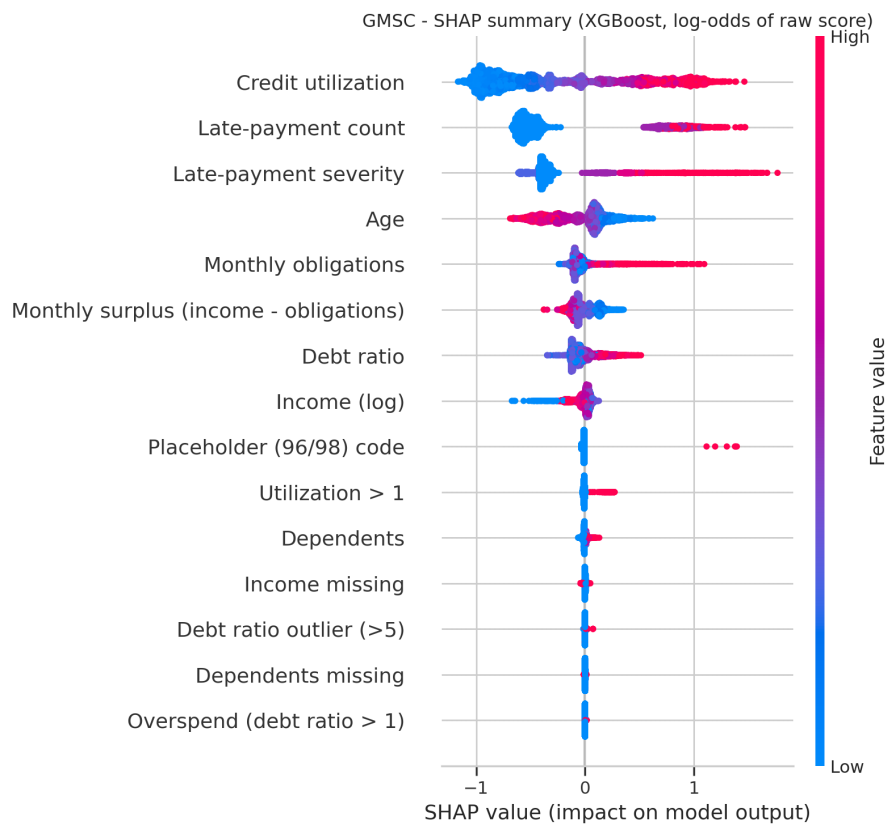
By risk driver (features of the same business meaning summed; SHAP is additive so this is exact):

Driver	Mean SHAP
Late payments	0.926
Credit utilization	0.679
Age	0.209
Income (incl. obligations, surplus)	0.167
Debt ratio	0.092
Dependents	0.014

By feature (top 10):

Feature	Mean SHAP
Credit utilization	0.672
Late-payment count	0.592
Late-payment severity	0.413
Age	0.209
Monthly obligations	0.094
Monthly surplus	0.094
Debt ratio	0.092
Income (log)	0.050
Placeholder (96/98) code	0.015
Utilization > 1	0.015





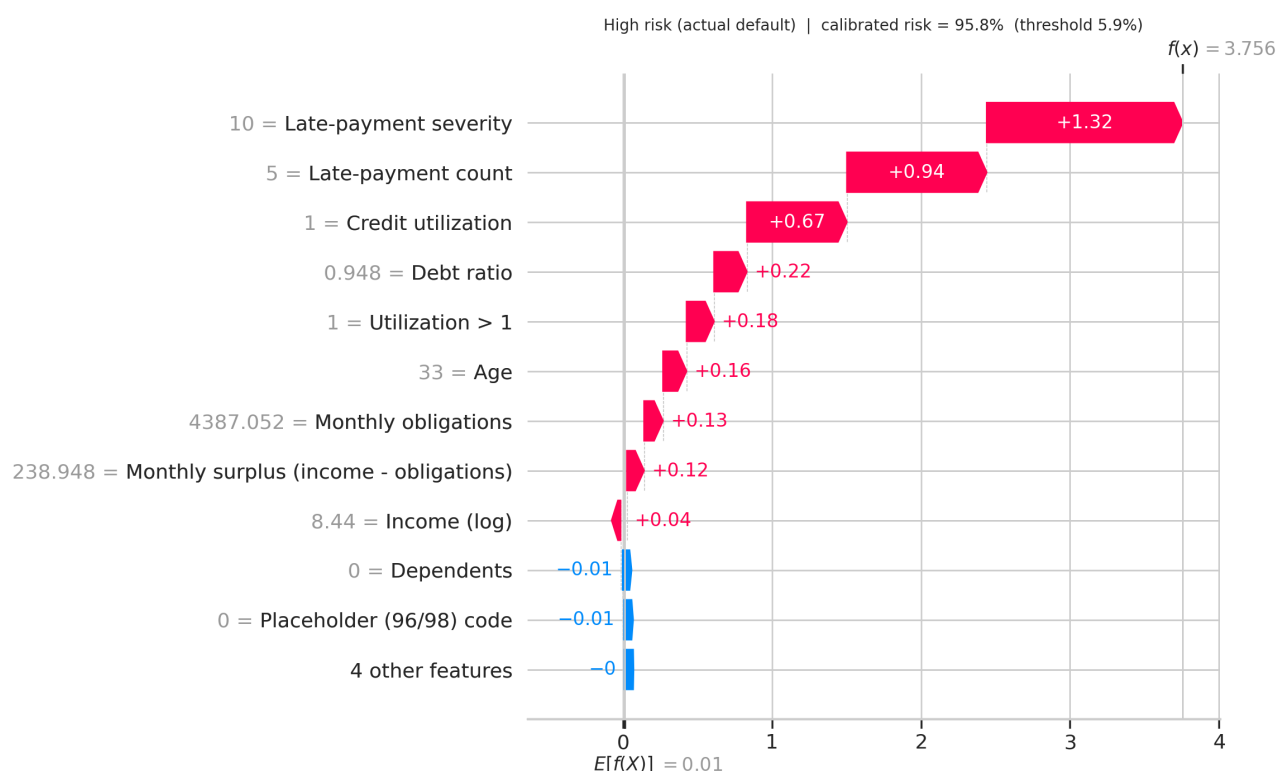
Findings

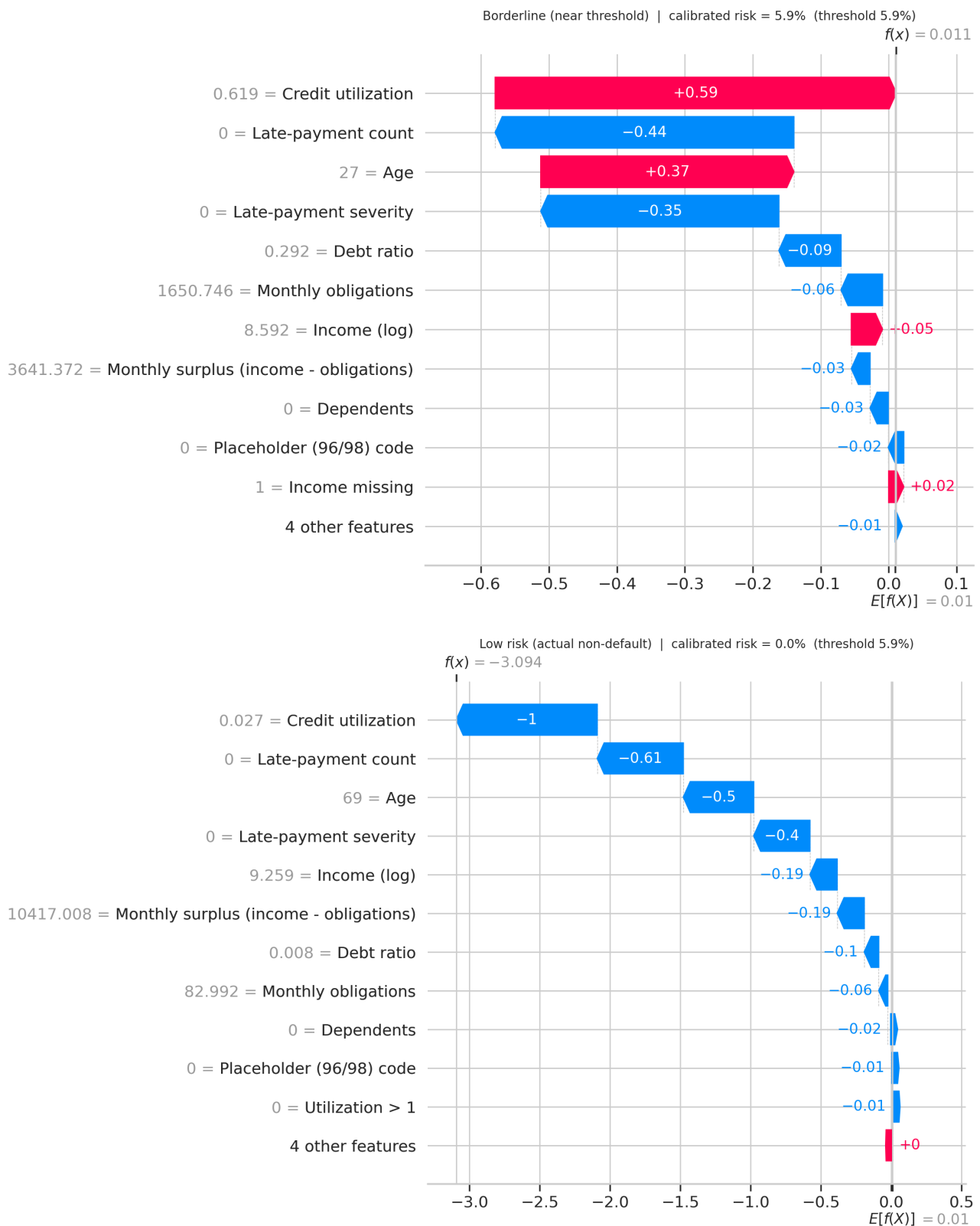
- The model is driven by **payment behavior** (late payments) and **credit utilization**, then **age**. This matches the EDA and the Logistic Regression coefficients.
- Higher utilization, more/longer late payments and younger age push risk up; the directions are monotonic and sensible.

- Income, debt ratio and dependents have a **small** effect. The 96/98 placeholder flag matters for very few customers (269) but raises their risk strongly.
- Reading caution:** income-related effects are split across four correlated features (log-income, obligations, surplus, missing flag), and the model uses them together. In the beeswarm, low log-income can look risk-reducing on its own; this is an artifact of correlated features. Use the **grouped** chart to explain income to users.

9.2 Individual explanations (waterfall)

Customer	Calibrated risk	Main reasons
High risk (actual default)	95.8%	Late-payment severity +1.32, late count +0.94, utilization +0.67, debt ratio +0.22
Borderline	5.9% (= threshold)	Utilization +0.59 and young age (27) +0.37 are offset by no late payments (−0.44, −0.35)
Low risk (actual non-default)	0.0%	Low utilization (0.03) −1.00, no late payments −0.61, age 69 −0.50





10. Prediction function and scenarios

`predict_risk(customer, MonthlyIncome, DebtRatio, NumberOfDependents)` takes **raw** customer data, applies the same feature engineering and train clipping, runs the XGBoost pipeline and the isotonic calibrator, and returns the calibrated risk and whether it is above the threshold. `compare_scenario` returns before/after/delta; `explain_customer` returns the risk and the top drivers (and the waterfall).

- **Consistency check:** on raw test rows the function reproduces the Part 8 test probabilities exactly (max difference = 0).
- **keep_obligations (default True):** $\text{DebtRatio} = \text{debt payments} / \text{income}$. When income changes and **DebtRatio** is not given, the function rescales **DebtRatio** so that the **dollar obligations stay constant**. Otherwise a higher income would silently imply higher debts.

Scenario sensitivity (2,341 test customers with valid income, mean base risk 7.0%):

Scenario	Mean change in risk	Customers whose risk changed	Flag flipped
Income +30% (obligations fixed)	−0.5 pp	65%	2.5%
Income +30% (DebtRatio unchanged)	−0.02 pp	59%	2.4%
DebtRatio × 0.5	−0.6 pp	58%	4.2%
Dependents → 0	−0.06 pp	11%	0.2%
Dependents + 2	+0.16 pp	31%	0.4%

Implications for the next stage (Scenarios / Recommendations):

- Directions are correct, but the effects are **small**: this model reacts mostly to late payments and utilization.
- Dependents are almost irrelevant; avoid building recommendations on them.
- Income scenarios must keep obligations fixed (see above), otherwise the effect is misleadingly near zero.
- The isotonic calibrator has only ~105 distinct output levels, so for a single customer a small change can give exactly 0 change (especially at very high risk). Show changes as ranges or averages, or compare the raw scores.

11. Deviations from the Week 2 spec and open points

#	Point	Status / recommendation
1	Clipping is done outside the pipeline (limits from the full train set, before CV)	Move it into a custom Winsorizer transformer inside the pipeline so limits are learned per fold
2	No separate validation set. Calibration and threshold use train OOF scores	Document as a deviation; the final test confirms the choices (Brier 0.0489, recall 0.82)
3	Earlier cells (Part 7) are superseded. They tuned XGBoost on features transformed with a preprocessor fit on the whole train set and without early stopping (CV AUC-PR 0.3969 / ROC-AUC 0.8615; 3-fold Optuna best 0.3961)	Remove or label as superseded; Section 6 is the valid result
4	Duplicates: 609 rows identical on all features may fall in both train and test	Small effect (0.4% of rows); mention in the report or deduplicate before the split
5	Feature code outside the artifact: joblib does not store functions	Put build_gmsc_features and the prediction functions in a .py module used by the app
6	XGBoost vs LR gain is small (+0.009 AUC-PR)	State the justification as in Section 6
7	EDA text error: cells 57 and 59 of the notebook say the default rate is <i>higher</i> when MonthlyIncome is missing; the numbers show the opposite (5.61% missing vs 6.95% present)	Fix the markdown (missingness still carries signal, so the flag is kept)

12. Saved artifacts

File	Content
gmsc_baseline_model.joblib, gmsc_preprocessor.joblib	Baseline LR pipeline and its preprocessor
gmsc_final_model.joblib	xgb_pipeline, calibrator, threshold, features, clip_lower, clip_upper, best_params
gmsc_train.csv, gmsc_test.csv, gmsc_target_features.csv	Split and engineered datasets
gmsc_shap_*.png	SHAP figures (6)